

ADR 0134 — Two censuses that must agree are checked against each other

- **Status:** Accepted — implemented, mutation-proved two ways failing differently
- **Date:** 2026-09-06
- **Issue:** #690
- **Extends:** [ADR 0119](#) — "a list of known sites is not a fix, because that list was already incomplete twice — the safety has to live in the value," applied one level up: not to a list of call sites, but to two independently hand-maintained *tables*, each individually complete, that must both change together
- **Supersedes / superseded by:** —

Context

`crates/git-vista-server/src/route_authz.rs` 's `ROUTE_AUTHZ` table classifies every route `main.rs` 's `api_router` registers by the authorization it must sit behind. `crates/git-vista-server/src/planner/contract_suite.rs` 's `POST` table (`KNOWN_POST_ROUTES` after this change) classifies every `POST` route as either a git write that must reach the planner, or an explicitly-argued non-git write. Both tables are individually well-guarded: each has its own test proving it agrees with `main.rs`, in both directions (nothing registered is unclassified, nothing classified has been quietly deleted). Neither table was ever wrong on its own.

The gap is that **adding a route requires updating both**, and nothing connects them. Building #584/PR#688's `POST /api/settings/token` proved this concretely: a `PostToolUse` hook (`.claude/hooks/route-census-check.sh`) fires on every edit to `route_authz.rs` or `main.rs` and runs `every_registered_route_is_classified` — so `ROUTE_AUTHZ` got the new route immediately, at edit time, loudly. The hook does not know `contract_suite.rs` 's table exists, so it never ran `every_git_write_route_reaches_the_planner`. That test's own failure only surfaced later, in CI's `M1.06` write `contract + job` — a job the local `cargo test -p git-vista-server --bins` check that ran alongside the edit cannot even compile, since `#[cfg(test)] mod contract_suite` (like `route_authz` itself) does not exist in a `--bins`-only build.

Grok found the identical shape the same day, one level up again: #695's `POST` route table said "funnel rows below" for three bisect routes, and the funnel array named none of them — a table naming a row that does not exist, passing silently. Three instances of "a list of known things is not a fix, it is a promise nobody checks" in one day (ADR 0119's original finding, #666/#660's independently-discovered third call sites, and now this) is what turns a one-off oversight into a pattern worth a named decision.

Decision

1. A third test reads both tables directly and asserts their `POST` route sets are identical

`route_authz_and_write_contract_agree_on_every_post_route` (`planner/contract_suite.rs`) computes the set of `POST`-method routes from `route_authz::ROUTE_AUTHZ` and the set of routes named in `KNOWN_POST_ROUTES` (mapping `KNOWN_POST_ROUTES` 's one path-less exception, `create_session`, to its real route `POST /api/session` so both sets are keyed the same way), and panics naming the exact route and which table is missing it if the two sets disagree. This is a genuine cross-check, not a third independent re-scan of `main.rs`: a third scanner would just be a third hand-maintained thing that could itself drift, which is exactly the failure mode this ADR exists to close, one level further out.

Both tables' visibility changed from private to `pub(crate)` to make this possible: `ROUTE_AUTHZ`, its `Authz` enum (required because it appears in `ROUTE_AUTHZ` 's element type, even though the new test destructures it with `_` and never names it), and the `route_authz` module itself in `main.rs`;

`KNOWN_POST_ROUTES`, hoisted out of `every_git_write_route_reaches_the_planner` into a module-level `const` in `contract_suite.rs` so a sibling test can read it without re-deriving it from a local binding.

2. The `PostToolUse` hook is extended, not replaced

`route-census-check.sh` now also runs the new meta-census test whenever `route_authz.rs`, `main.rs`, or `contract_suite.rs` changes (the third path added because the hook's whole point is to catch a table drifting the moment *either side* is edited, not only when the router is). This is the fix for the actual failure mode #584 hit: the local, edit-time signal now covers both tables' relationship to each other, not just one table's relationship to `main.rs`.

3. A single source-of-truth table (the issue's other proposed direction) was not built

The issue offered three directions: a single table both classifications derive from, this meta-census, or a doc-comment cross-reference. The single-source table was rejected for this change:

`ROUTE_AUTHZ` and `KNOWN_POST_ROUTES` classify different things for different reasons (security posture vs. planner-funnel membership) and are read by tests with different shapes and different failure messages tuned to what a maintainer needs to do next; merging them into one row-per-route table with two classification columns would work, but is a larger, riskier refactor of two files with a combined 641 + 7500+ lines, undertaken to fix a discoverability gap that the meta-census closes just as completely at a fraction of the risk. Nothing here forecloses that direction later, the same posture ADR 0114 took toward its own declined alternative.

The doc-comment-only direction (weakest of the three, per the issue's own framing) was rejected outright: a comment is exactly the kind of fix that "looks read" without being enforced, the same shape ADR 0119 already ruled out for a call-site list.

Consequences

- A route added to one table and not the other now fails in the same test binary as both tables, by name, rather than only in whichever CI job happens to exercise the table nobody thought to check.
- The hook now catches the exact failure #584/PR#688 hit, at edit time, not merge time — closing the discoverability gap this issue was filed over.
- `ROUTE_AUTHZ`, `Authz`, and `KNOWN_POST_ROUTES` are `pub(crate)` now, purely so this cross-check test can read them; nothing outside `crates/git-vista-server`'s own test code is expected to use the wider visibility, and both are still `#[cfg(test)]`-gated at their module root.
- The next hand-maintained pair with the same shape (two tables, one router, no link) should get the same treatment rather than a bespoke fix — this meta-census pattern, not a single-source refactor, is now this project's default answer to "two lists must agree."

Mutation proof

Manual, two-way (`failure-atlas`'s `mutation_check` was globally unavailable at the time — a containment bug, "temp base /tmp is inside the git work tree," confirmed across two other lanes the same day, not fixable by retrying):

arm	mutation	mutated result
a route present in <code>ROUTE_AUTHZ</code> but missing from <code>KNOWN_POST_ROUTES</code>	removed the <code>/api/rescan</code> entry from <code>KNOWN_POST_ROUTES</code> (still classified in <code>ROUTE_AUTHZ</code>)	caught: <code>route_authz_and_write_contract_agree_on_every_post_rout</code> panics naming <code>/api/rescan</code> as present in <code>ROUTE_AUTHZ</code> but missing from <code>KNOWN_POST_ROUTES</code>
a route present in <code>KNOWN_POST_ROUTES</code> but missing from <code>ROUTE_AUTHZ</code>	removed the <code>("/api/rescan", Method::POST, ...)</code> entry from <code>ROUTE_AUTHZ</code> (still classified in <code>KNOWN_POST_ROUTES</code>)	caught: the same test panics on the opposite branch, naming <code>/api/rescan</code> as present in <code>KNOWN_POST_ROUTES</code> but missing from <code>ROUTE_AUTHZ</code> — a genuinely different assertion path (<code>contract_posts.difference(&authz_posts)</code> rather than <code>authz_posts.difference(&contract_posts)</code>)

Both caught, both reverted; `git diff` shows no trace of either mutation afterward.